

Appunti del 05-03-26 – Mattina

PROGETTO BLOGGER – Bella Copia

1. Obiettivi e Approccio

- Imparare a gestire la complessità.
- Analizzare i requisiti: capire chi usa il sistema e cosa fa.

2. Descrizione della Piattaforma

Vogliamo creare una piattaforma da offrire a chi vuole aprire un blog, con AI integrata per effettuare una "censura preventiva" — ovvero moderare i contenuti del sito che ospitiamo.

Non si può perdere tempo a cercare la soluzione migliore: quando la si trova, è già obsoleta. L'approccio corretto è scegliere un punto di arrivo raggiungibile e arrivarci.

3. Attori del Sistema

3.1 Visitor (Visitatore)

- Può vedere la Home Page con gli ultimi articoli pubblicati.
- Può cercare per blogger, blog o argomento (modalità di ricerca ancora da definire).
- Per pubblicare contenuti deve registrarsi come "autore".

3.2 Blogger

- Estende il Visitor (eredita le sue funzionalità).
- È sia un attore (usa il sistema) sia un'entità del data model.
- Può gestire i propri blog.
- Non è obbligato a cambiare la password.

3.3 Admin

- Non estende il Visitor/Blogger.
- Può gestire i blog di altri utenti.
- È obbligato a cambiare la password ogni due settimane.

4. Data Model

4.1 Entità USER

Gli utenti devono essere identificabili per ragioni di responsabilità penale. Campi richiesti:

- ID
- First Name (Nome)
- Last Name (Cognome)
- DOB – Date of Birth (Data di nascita, eventualmente con documento d'identità)
- Nickname
- Email
- Password
- Role: BLOGGER | ADMIN

4.2 Entità BLOG

Un blog è un insieme di post con tema comune. Ogni blog è personalizzabile (palette colori, template). Campi:

- ID
- Title (univoco)
- Description (String)
- Visibility: PUBLIC | PRIVATE (privato = su invito)
- Palette (colori, 2/3 scelte)
- Template (2/3 scelte)

Relazioni:

- Un User gestisce N Blog (relazione 1 a N).
- N Blog usano 1 Template.
- N Blog condividono lo stesso insieme di Palette colori.

4.3 Entità BLOG POST

Un Blog contiene N BlogPost. Ogni post può avere una policy sui commenti: l'autore può decidere se accettarli o meno sul singolo post.

4.4 Entità COMMENT

Un BlogPost può contenere N Commenti. I commenti esistono solo se il relativo BlogPost esiste.

4.5 Tags (Argomenti)

I blog e i post possono essere associati a tag/argomenti per la ricerca e la categorizzazione.

5. Composizione e Integrità Referenziale

5.1 Composizione (sensu strictu)

La composizione significa che l'entità figlia non può esistere senza l'entità padre. Se il padre viene eliminato, anche i figli vengono eliminati a cascata. Nel diagramma UML questo si mappa direttamente su ON DELETE CASCADE nel database.

5.2 Integrità Referenziale

Vincolo del database che garantisce che le relazioni tra tabelle rimangano consistenti.

- Blog Post → Blog: la chiave esterna blog_id in BlogPost deve sempre riferirsi a un Blog esistente.
- Comment → Blog Post: la chiave esterna post_id in Comment deve sempre riferirsi a un BlogPost esistente.

5.3 Operazioni Critiche

INSERT – prima si crea il padre, poi il figlio:

- Prima si crea il Blog, poi il BlogPost, infine il Comment.

DELETE – usando CASCADE:

- Eliminare il Blog → si eliminano automaticamente tutti i suoi BlogPost e, di conseguenza, tutti i Comment.
- Eliminare il BlogPost → si eliminano automaticamente tutti i suoi Comment.

UPDATE – se si aggiorna la chiave primaria del padre (operazione rara), anche la FK del figlio deve aggiornarsi (ON UPDATE CASCADE).

6. Note di Implementazione (Spring Boot)

6.1 Annotazioni principali

- @Data (Lombok) → genera automaticamente toString(), getter e setter.
- @Valid → annotazione Jakarta Bean Validation; valida automaticamente i dati in ingresso (tipicamente il body di una richiesta POST/PUT).

- `@RequestBody` → dice a Spring di leggere il corpo (body) della richiesta HTTP e convertirlo automaticamente in un oggetto Java (deserializzazione JSON → oggetto).

6.2 ResponseEntity<T>

Permette di controllare completamente la risposta HTTP: body, status code e headers.

Codice	Significato	Metodo
200 OK	Operazione riuscita	<code>ResponseEntity.ok(user)</code>
201 Created	Risorsa creata	<code>ResponseEntity.status(201).body(savedUser)</code>
204 No Content	Nessun contenuto (es. DELETE)	<code>ResponseEntity.noContent().build()</code>
400 Bad Request	Dati non validi	<code>ResponseEntity.badRequest().body("...")</code>
404 Not Found	Risorsa non trovata	<code>ResponseEntity.notFound().build()</code>
500 Internal Error	Errore del server	<code>ResponseEntity.internalServerError().body("...")</code>

6.3 Enum Role

```
public enum Role {
    BLOGGER,
    ADMIN
}
```

7. Scadenza e Obiettivo Immediato

Scadenza: 06/03/2026 – ore 13:00

Consegnare una pagina web con un Blog di Prova inserito manualmente dall'admin, con dati di prova, che dia un'idea concreta di come funzionerà il tutto.

Il sistema di login e la maggior parte delle API non saranno ancora presenti.

7.1 Specifiche tecniche

- Solo lettura: API GET `/api/voices/blogs/1`
- Percorso per titolo: `/voices/blogs/la_strada_e_i_canti` (il titolo del blog deve essere univoco)

- Il frontend Angular raggiungerà il componente BlogPage tramite questo percorso.

Principio guida: scegliere un punto di arrivo raggiungibile, crederci e arrivarci — non costruire tutto il sistema in una volta.